

Инструкция по запуску заданий лабораторной работы 4

Этот PDF содержит практические инструкции по запуску всех заданий ЛР4: где именно запускать каждую ветку работы, как открывать проекты в Qt Creator, как запускать Linux-ветку через WSL на Windows, как собирать bonus-ветку cdecl32 и какие онлайн-ресурсы можно использовать для проверки и демонстрации.

1. Общий принцип организации папок

Каждое задание ЛР4 у вас оформлено как отдельный проект. Обычно структура выглядит так:

- L4_1_ms64, L4_1_sysv64, L4_1_cdecl32
- L4_2_ms64, L4_2_sysv64, L4_2_cdecl32
- L4_3_ms64, L4_3_sysv64, L4_3_cdecl32
- L4_4_cpp, L4_4_ms64, L4_4_sysv64, L4_4_cdecl32
- L4_5_ms64, L4_5_sysv64, L4_5_cdecl32
- L4_6_ms64, L4_6_sysv64, L4_6_cdecl32

Это значит, что нельзя открывать весь архив как один проект. Нужно открывать или собирать отдельно именно ту папку, которая соответствует конкретному заданию и конкретному соглашению о вызовах.

2. Что где запускать

- *_ms64 — запускать на Windows 64. Это ветка под соглашение Microsoft 64.
- *_sysv64 — запускать в Linux-среде. Если у вас только Windows, удобнее всего использовать WSL + Ubuntu.
- *_cdecl32 — запускать как 32-битную x86-сборку с флагом -m32. Лучше делать это тоже через WSL, но с установленной multilib-поддержкой.
- L4_4_cpp — это обычная C/C++ версия задания 4, её можно запускать и на Windows через Qt Creator.

Самая удобная и безопасная схема такая: Microsoft 64 выполнять локально в Qt Creator на Windows; System V amd64 выполнять через WSL; cdecl32 выполнять через WSL с multilib.

3. Как запускать проекты *_ms64 в Qt Creator

Для ветки Microsoft 64 удобнее всего использовать Qt Creator. Поскольку в ваших папках уже есть файлы project.pro, создавать новые проекты не нужно.

- Откройте Qt Creator.
- Нажмите File → Open File or Project.
- Выберите нужный файл project.pro, например: L4_1_ms64/project.pro.
- На экране настройки выберите kit Desktop Qt 6.7.3 MinGW 64-bit.
- Нажмите Configure Project.

- Далее нажмите Build, затем Run.

Точно так же открываются и запускаются остальные Windows-проекты: L4_2_ms64, L4_3_ms64, L4_4_cpp, L4_4_ms64, L4_5_ms64, L4_6_ms64.

Важно: каждый проект открывается отдельно. Не нужно держать весь архив как один общий проект.

4. Как запускать ветку *sysv64, если у вас есть только Windows

Если физически у вас только Windows, это не мешает выполнить Linux-ветку. Для этого лучше всего использовать WSL (Windows Subsystem for Linux) и установить Ubuntu.

Шаг 1. Установите WSL из PowerShell, запущенного от имени администратора:

```
wsl --install
```

После установки и перезагрузки откройте Ubuntu и установите базовый набор инструментов для сборки:

```
sudo apt update  
sudo apt install build-essential
```

Теперь вы можете перейти в папку с проектом через путь /mnt/c/... и собрать проект обычной консольной командой.

Пример перехода к папке проекта:

```
cd /mnt/c/Users/БАШЕ_ИМЯ/Documents/lab4/L4_1_sysv64
```

Дальше используйте одну из следующих схем сборки.

5. Базовые шаблоны команд сборки

Тип проекта определяется набором файлов в папке.

- main.cpp + unit.S — обычно это задания Л4 №1 и Л4 №2.
- main.S — обычно это задания Л4 №3, Л4 №4 asm и Л4 №5.
- main.S + unit.cpp — обычно это Л4 №6.

Команды сборки для Linux-ветки (*sysv64):

```
# Если в папке main.cpp + unit.S  
g++ main.cpp unit.S -o app  
./app
```

```
# Если в папке только main.S  
g++ main.S -o app  
./app
```

```
# Если в папке main.S + unit.cpp  
g++ main.S unit.cpp -o app  
./app
```

6. Как запускать ветку *cdecl32

Ветка cdecl32 — это уже 32-битная x86-сборка. Поэтому при компиляции обязательно нужен флаг -m32. Но одного флага недостаточно: в системе должны быть 32-битные библиотеки, иначе линковка не пройдёт.

В WSL/Ubuntu установите multilib-пакеты:

```
sudo apt update
sudo apt install gcc-multilib g++-multilib
```

После этого команды сборки выглядят так:

```
# main.cpp + unit.S
g++ -m32 main.cpp unit.S -o app
./app
```

```
# main.S
g++ -m32 main.S -o app
./app
```

```
# main.S + unit.cpp
g++ -m32 main.S unit.cpp -o app
./app
```

Если после этого всё равно появляется ошибка линковки, значит в системе всё ещё не хватает 32-битных зависимостей или multilib установлен неполно.

7. Как запускать каждое задание отдельно

Л4 №1

- Обычно содержит: main.cpp и unit.S.
- Windows-ветка: открыть L4_1_ms64/project.pro в Qt Creator и запустить.
- Linux-ветка: перейти в L4_1_sysv64 и выполнить `g++ main.cpp unit.S -o app`, затем `./app`.
- cdecl32: перейти в L4_1_cdecl32 и выполнить `g++ -m32 main.cpp unit.S -o app`, затем `./app`.
- Обычно программа просит ввести два целых числа.

Л4 №2

- Обычно содержит: main.cpp и unit.S.
- Windows-ветка: открыть L4_2_ms64/project.pro.
- Linux-ветка: `g++ main.cpp unit.S -o app` и затем `./app`.
- cdecl32: `g++ -m32 main.cpp unit.S -o app` и затем `./app`.
- Обычно программа просит ввести два числа типа double.

Л4 №3

- Обычно содержит только main.S.
- Windows-ветка: открыть L4_3_ms64/project.pro.
- Linux-ветка: `g++ main.S -o app`, затем `./app`.
- cdecl32: `g++ -m32 main.S -o app`, затем `./app`.
- Ввода обычно нет — программа просто печатает текст.

Л4 №4 — C/C++ версия

- Папка: L4_4_cpp.
- Файл: main.cpp.
- Можно запускать в Qt Creator или через команду `g++ main.cpp -o app`.
- Программа обычно вводит 5 значений: i16, i32, i64, f32, f64.

Л4 №4 — asm версия

- Обычно содержит только main.S.
- Windows-ветка: открыть L4_4_ms64/project.pro.
- Linux-ветка: `g++ main.S -o app`, затем `./app`.
- cdecl32: `g++ -m32 main.S -o app`, затем `./app`.
- Программа обычно тоже вводит 5 значений и затем их печатает.

Л4 №5

- Обычно содержит только main.S.
- Запускается так же, как Л4 №4 asm.
- Разница не в запуске, а в реализации стека: здесь классический пролог/эпилог.

Л4 №6

- Обычно содержит main.S и unit.cpp.
- Windows-ветка: открыть L4_6_ms64/project.pro.
- Linux-ветка: `g++ main.S unit.cpp -o app`, затем `./app`.
- cdecl32: `g++ -m32 main.S unit.cpp -o app`, затем `./app`.
- Ввод, как правило, не нужен: программа сама вызывает функцию с 8 параметрами.

8. Онлайн-ресурсы, если нужна проверка или запуск без локальной Linux-среды

- Compiler Explorer (<https://godbolt.org>) — лучший вариант для быстрого тестирования asm и mixed-проектов. Подходит для main.cpp + unit.S, main.S + unit.cpp, а также для просмотра ассемблера.
- OnlineGDB (https://www.onlinegdb.com/online_gcc_assembler) — удобен для однофайловых программ на ассемблере, например main.S.
- Для ветки cdecl32 на онлайн-платформах иногда можно указать дополнительный флаг `-m32`, но это зависит от окружения и не всегда работает стабильно.

Онлайн-платформы удобно использовать для быстрой проверки, но для сдачи надёжнее иметь локальный запуск: Windows-ветку на Qt Creator, Linux-ветку через WSL.

9. Самые частые ошибки

- Открывают не отдельный project.pro, а пытаются импортировать весь архив как один проект.
- Пытаются запускать ветку sysv64 через Windows kit — это неверно с точки зрения соглашения о вызовах.
- Для cdecl32 забывают флаг -m32.
- Для cdecl32 не устанавливают multilib, из-за чего сборка не линкуется.
- В mixed-проектах забывают второй файл: unit.S или unit.cpp.
- Путают L4_4_cpp и L4_4_* — первая папка содержит обычную C/C++ версию, а остальные — ассемблерные.

10. Быстрый чек-лист перед демонстрацией преподавателю

- Для каждого задания подготовлены две обязательные ветки: ms64 и sysv64.
- Если делаете бонус, отдельно проверена ветка cdecl32.
- Запуск Windows-ветки показан через Qt Creator.
- Запуск Linux-ветки можно показать через WSL или, как минимум, продемонстрировать команды сборки.
- Для заданий №4 и №5 подготовлены рисунки стека.
- Для каждого проекта понятно, какие файлы участвуют в сборке: main.cpp + unit.S, main.S, либо main.S + unit.cpp.

11. Краткая памятка-команды

Windows (через Qt Creator)

Открыть project.pro → Configure Project → Build → Run

Linux / sysv64

g++ main.cpp unit.S -o app && ./app

g++ main.S -o app && ./app

g++ main.S unit.cpp -o app && ./app

cdecl32

g++ -m32 main.cpp unit.S -o app && ./app

g++ -m32 main.S -o app && ./app

g++ -m32 main.S unit.cpp -o app && ./app

Готово. Этот документ можно использовать как отдельную инструкцию по запуску всех заданий ЛР4.